LOADPILOT — PLATEFORME DE TEST DE CHARGE

État d'avancement
Module Gestion des scénarios

Ce qui a été réalisé, du début à la fin, et le statut vérifié de chaque fonctionnalité, avec ses preuves dans le code, dans Git et dans les tests exécutés

FONCTIONNALITÉS
17 ✅ · 3 🟡 · 1 ❌

TESTS BACKEND
652 / 652 réussis (02/10, 22:11)

TESTS FRONTEND
99 / 99 réussis (02/10, 21:08)

ENDPOINTS DU MODULE
38 (scénarios, étapes, tests, exécutions)

MIGRATIONS LIQUIBASE
20 au total, dont 017 à 020 pour ce module

GIT
Dernier commit le 26/09 ; travail ultérieur non commité

Sommaire

1. Synthèse
2. Méthode et sources
3. Historique Git et chronologie
4. Statut des fonctionnalités
5. Endpoints REST du module
6. Base de données (migrations)
7. Résultats des tests
8. Ce qui manque, risques et prochaines étapes

1. Synthèse

Le module Gestion des scénarios couvre la création, la modification, le test et l'exécution en charge des scénarios HTTP de LoadPilot. Après le dernier commit (26/09/2026), il a été étendu par étapes. On a ajouté l'assistant de création avec brouillons, l'import CSV, le test détaillé des étapes, le statut INACTIF, les actions groupées, le système de variables, le suivi d'exécution en direct et la page de modification complète.

17
fonctionnalités ✅ terminées

3
fonctionnalités 🟡 partielles

1
point ❌ non réalisé (commit Git)

751
tests exécutés, 0 échec (652 + 99)

Vue d'ensemble des statuts

#	Fonctionnalité	Statut
A	CRUD des scénarios et des étapes (socle)	✅ **Terminé**
B	Suppression avec impact, archivage, restauration	✅ **Terminé**
C	Assistant de création en étapes	✅ **Terminé**
D	Brouillons (DRAFT), « Reprendre », protection en quittant	✅ **Terminé**
E	Bouton « Suivant » bloqué, même validation côté front et back	✅ **Terminé**
F	Import de fichier CSV (aperçu, contrôles, modèle)	✅ **Terminé**
G	Test détaillé d'une étape ou d'un scénario (« Tester »)	✅ **Terminé**
H	Liste des scénarios avec pagination et tri serveur	✅ **Terminé**
I	Actions groupées (archiver, restaurer, supprimer, statut)	✅ **Terminé**
J	Statut INACTIF fonctionnel (planifications en pause)	✅ **Terminé**
K	Moteur de variables (capture, CSV, variables de scénario, contexte par VU)	✅ **Terminé**
L	Onglet « Paramètres » (path/query) et variables de scénario dans l'UI	✅ **Terminé**
M	Étape « Charge & résumé »	✅ **Terminé**
N	Suivi d'exécution en direct, annulation, fenêtre de résultat	✅ **Terminé**
O	Page de modification `/scenarios/:id/edit` et `PUT /full`	✅ **Terminé**
P	« Ajouter une étape » dans un panneau latéral (StepDrawer)	✅ **Terminé**
Q	Message d'authentification masqué en cas de succès dans « Tester »	✅ **Terminé**
R	Traçabilité VU / itération / URL finale par résultat	🟡 **Partiel**
S	Options avancées d'étape (timeout, think time, redirections) dans l'éditeur	🟡 **Partiel**
T	Annulation d'une exécution : conservation des résultats partiels	🟡 **Partiel**
U	Travail versionné dans Git (commit)	❌ **Non réalisé**

Légende : ✅ Terminé, implémenté et couvert par des tests qui passent · 🟡 Partiel, en partie implémenté, le manque est décrit · ❌ Non réalisé.

2. Méthode et sources

Ce rapport ne contient que des éléments vérifiables. Aucun fichier de code n'a été modifié pour le produire.

Source	Ce qui en a été extrait
Historique Git	`git log` (commits, auteurs, dates, nombre de fichiers), `git status` et `git diff --stat` (fichiers modifiés ou non suivis), dates de modification des fichiers non suivis.
Code actuel	Annotations `@GetMapping`, `@PostMapping`, etc. des contrôleurs ; fichiers de migration Liquibase ; classes et composants cités dans chaque fiche ; recherches ciblées pour confirmer les manques (par exemple l'absence de champ timeout dans `StepEditor.tsx`).
Tests backend	`mvn -o test` lancé le 02/10/2026, terminé à 22:11:02 (+01:00) en 2 min 46 s : **652 tests, 0 échec, 0 erreur, 0 ignoré**, BUILD SUCCESS.
Tests frontend	Vitest 2.1.9, rapport JSON du 02/10/2026 à 21:08 : **99 tests sur 14 fichiers, 99 réussis**.
Vérifications réelles	Scripts lancés contre le backend local (port 8080) et l'API de démonstration banking-test-api (port 8888). Le mot de passe passe par une variable d'environnement et n'est pas reproduit ici.

> Aucun identifiant, mot de passe ni secret n'apparaît dans ce document.

3. Historique Git et chronologie

3.1 Commits existants

Commit	Date	Message	Volume
`f04f935`	09/09/2026 21:11 (+01:00)	Add frontend and backend	90 fichiers, +40 585 lignes
`3003211`	26/09/2026 16:37 (UTC)	Finalize LoadPilot ADRIA migration	371 fichiers, +33 037 / −15 582

Ces deux commits forment le socle du projet : CRUD des scénarios et des étapes, moteur d'exécution, planifications et rapports.

3.2 Travail non commité (état de l'arbre de travail)

142
entrées dans `git status`

74
fichiers suivis modifiés

+3 527 / −1 477
lignes sur les fichiers suivis

68
fichiers ou dossiers non suivis (nouveaux)

> Tout le travail décrit dans la chronologie ci-dessous est **hors Git** : aucun commit depuis le 26/09/2026.

3.3 Chronologie, d'après les dates des nouveaux fichiers

Période	Travail réalisé
01/10, 14 h	Historique des suppressions et archivage (migration 017)
01/10, 16:00 – 16:23	Test détaillé des étapes et scénarios (StepTestRunner, TestRunController, TestRunPanel)
01/10, 18:11 – 23:25	Brouillons, validation partagée, import CSV (migration 018)
02/10, 02:05 – 02:21	Statut INACTIF, actions groupées, pagination serveur (migration 019)
02/10, 04:17 – 04:46	Système de variables, onglet Paramètres, variables de scénario (migration 020)
02/10, 19:25 – 19:39	Étape « Charge & résumé », suivi d'exécution en direct
02/10, 19:54 – 20:07	Page de modification et panneau StepDrawer
02/10, 21:08 et 22:11	Exécution complète des tests frontend puis backend pour ce rapport

3.4 Taille des principaux fichiers

Fichier	Lignes
`perftest-frontend/src/pages/scenario-wizard/ScenarioWizard.tsx`	1 124
`perftest-frontend/src/pages/Scenarios.tsx`	1 104
`backend/…/service/execution/StepTestRunner.java`	541
`perftest-frontend/src/pages/scenario-wizard/StepEditor.tsx`	535
`backend/…/service/execution/HttpClientExecutionEngine.java`	480
`backend/…/service/impl/ScenarioServiceImpl.java`	379

4. Statut des fonctionnalités

Chemins abrégés : `back/` = `backend/src/main/java/com/loadpilot/backend/` ; `front/` = `perftest-frontend/src/` ; `wiz/` = `front/pages/scenario-wizard/`.

A. CRUD des scénarios et des étapes (socle) — ✅ Terminé

Preuve	`back/controller/ScenarioController.java`, `StepController.java`, `ScenarioServiceImpl`, mappers MapStruct ; présent depuis les commits `f04f935` et `3003211`.
Endpoints	`GET/POST /api/scenarios`, `GET/PUT/DELETE /api/scenarios/{id}`, `GET/POST /api/steps`, `GET/PUT/DELETE /api/steps/{id}`
Tests	ScenarioControllerTest (22), StepControllerTest (28), ScenarioRepositoryTest (6), StepRepositoryTest (6), RequestValidationTest (22) : tous réussis.
Manque	Rien.

B. Suppression avec impact, archivage, restauration — ✅ Terminé

Preuve	Migration `017-deletion-history-and-archiving.xml` ; `DeletionServiceImpl` (dont `scenariosImpact`).
Endpoints	`GET /api/scenarios/{id}/deletion-impact`, `POST /{id}/archive`, `POST /{id}/restore`, `POST /bulk-deletion-impact`
Tests	DeletionArchivingPagingTest (7) réussi.
Manque	Rien.

C. Assistant de création en étapes — ✅ Terminé

Preuve	`wiz/ScenarioWizard.tsx`, `wizardModel.ts`, `StepEditor.tsx`, `scenarioWizard.css` ; route `scenarios/new` dans `front/App.tsx`.
Tests	ScenarioWizard.test (5), validation.test (10) réussis.
Manque	Rien sur le plan fonctionnel. Le rendu visuel n'a pas été contrôlé dans un navigateur (voir section 8).

D. Brouillons (DRAFT), « Reprendre », protection en quittant — ✅ Terminé

Preuve	Enum `ScenarioStatus` (DRAFT, ACTIVE, INACTIVE) ; migration 018 (colonnes `wizard_step` et `draft_state`) ; `ScenarioServiceImpl` (publish, deleteDraft) ; `wiz/draftModel.ts` ; `useBlocker` dans `ScenarioWizard.tsx` avec fenêtre de confirmation.
Endpoints	`POST /api/scenarios/{id}/publish`, `DELETE /api/scenarios/{id}/draft`
Tests	ScenarioDraftControllerTest (4) réussi. `ExecutionTransactionHelper` refuse d'exécuter un brouillon.
Manque	Rien.

E. Bouton « Suivant » bloqué, même validation côté front et back — ✅ Terminé

Preuve	`back/service/validation/StepRules.java` (URL, en-têtes, corps JSON, capture, paramètres), dont les règles sont reprises côté front dans `wizardModel.ts`. Le bloc « Pour continuer » a été retiré : le bouton reste désactivé et l'erreur s'affiche sous le champ une fois celui-ci touché.
Tests	ScenarioWizard.test (5), validation.test (10) réussis.
Manque	Rien.

F. Import de fichier CSV ✅ Terminé

Preuve	Backend : `back/service/execution/CsvDataSource.java`, réécrit (parseur RFC 4180, BOM, séparateur `;` ou `,`, guillemets, validation) ; migration 018 (`csv_data` en VARCHAR(1000000)). Frontend : `wiz/CsvPanel.tsx` et `csvModel.ts` (aperçu, alertes de cohérence, modèle téléchargeable). Exemples réels : `docs/samples/banking-test-api-users.csv` et `docs/samples/banking-variables.csv`.
Tests	CsvDataSourceTest (12), csvModel.test (7) réussis ; scénario banking-test-api exécuté avec ce CSV.
Manque	Les valeurs CSV ne sont pas échappées pour le JSON (voir section 8).

G. Test détaillé d'une étape ou d'un scénario (« Tester ») ✅ Terminé

Preuve	`back/service/execution/StepTestRunner.java` (flux NDJSON via `ResponseBodyEmitter`, masquage des données sensibles avec `TestRunMasking`, corps limité à 100 Ko, délai global de 60 s) ; `TestRunPlanner` (scénario, étape, version non enregistrée jusqu'à `upToStep`) ; `wiz/TestRunPanel.tsx`, `TestResultCard.tsx`, `testRunModel.ts`.
Endpoints	`POST /api/scenarios/{id}/test-run` (avec `stream=true` en option), `POST /api/steps/{id}/test-run` (avec `stream=true` en option)
Tests	StepTestRunnerTest (13), TestRunControllerTest (4), testRun.test (6) réussis.
Manque	Rien.

H. Liste des scénarios avec pagination et tri serveur ✅ Terminé

Preuve	`GET /api/scenarios` accepte des paramètres de page ; filtres dans `back/specification/ScenarioSpecifications` ; tableau paginé dans `front/pages/Scenarios.tsx`.
Tests	Scenarios.test (7), Pagination.test (4), DeletionArchivingPagingTest (7) réussis.
Manque	Rien.

I. Actions groupées ✅ Terminé

Preuve	`back/service/impl/ScenarioBulkService`, qui renvoie un résultat par identifiant (`BulkActionResponse`) ; DTO `ScenarioBulkRequest` ; barre d'actions groupées dans `Scenarios.tsx`.
Endpoints	`POST /api/scenarios/bulk-archive`, `/bulk-restore`, `/bulk-delete`, `/bulk-status`, `/bulk-resume-schedules`, `/bulk-deletion-impact`
Tests	ScenarioStatusAndBulkControllerTest (5), Scenarios.test (7) réussis.
Manque	Rien.

J. Statut INACTIF fonctionnel ✅ Terminé

Preuve	`ScenarioStatusService` ; migration 019 (colonne `scheduled_execution.paused_by_scenario`) ; la requête des planifications à lancer ne retient que les scénarios ACTIVE ; `ExecutionTransactionHelper` refuse d'exécuter un scénario INACTIF.
Endpoints	`POST /api/scenarios/{id}/activate`, `/deactivate`, `/resume-schedules`
Tests	ScenarioStatusAndBulkControllerTest (5), statusMapping.test (10) réussis.
Manque	Rien.

K. Moteur de variables

✅ **Terminé**

Preuve	`back/service/execution/VariableContext` (un contexte par VU, captures remises à zéro à chaque itération), `VariableSources` (distribution CSV par VU ou par itération, enum `CsvDistribution`), `RequestBuilder` (priorité capture > CSV > variable de scénario > `${authToken}`, encodage de l'URL, échec avant l'envoi si une variable est introuvable), `back/service/validation/ScenarioVariables`.
Tests	RequestBuilderTest (8), EngineVariablesTest (4), VariableResolverTest (7), VariablesControllerTest (1), HttpClientExecutionEngineTest (48) réussis. Exécution réelle sur banking-test-api : 59 DELETE sur 59 `transferId` distincts, aucun 404 ni 409 inattendu.
Manque	JSONPath limité aux chemins pointés (pas d'index de tableau) ; pas d'échappement JSON des valeurs injectées (voir section 8).

L. Onglet « Paramètres » et variables de scénario dans l'UI

✅ **Terminé**

Preuve	Migration 020 (colonnes `step.path_params`, `step.query_params`, `scenario.variables`, `scenario.csv_distribution`) ; `StepRequest` et `StepResponse` (+pathParams, queryParams) ; `wiz/StepEditor.tsx` (onglet Paramètres, `VariableInput`), `ScenarioVariablesPanel.tsx`, `variablesModel.ts`.
Tests	variables.test (8) réussi.
Manque	Rien.

M. Étape « Charge & résumé »

✅ **Terminé**

Preuve	`wiz/LoadSummaryPanel.tsx` ; avertissements de charge (`loadWarnings`) dans `wizardModel.ts`.
Tests	load.test (3) réussi.
Manque	Rien.

N. Suivi d'exécution en direct

✅ **Terminé**

Preuve	Backend : `LiveStats` (compteurs `LongAdder`, débit calculé sur 5 s) rattaché à l'exécution en cours ; `ExecutionServiceImpl.getStatus` (progression, statistiques en direct) ; `ExecutionStatusResponse` enrichi. Frontend : `front/pages/ExecutionLive.tsx`, `ExecutionMonitorContext.tsx` (monté dans `MainLayout`, le suivi continue quand on change de page), route `executions/live/:id`.
Endpoints	`GET /api/executions/{id}/status` (interrogé chaque seconde), `POST /api/executions/{id}/cancel`
Tests	ExecutionLiveStatusTest (1), LiveStatsTest (2), ExecutionLive.test (5), ExecutionControllerTest (23) réussis.
Manque	Les résultats partiels sont perdus à l'annulation (voir T).

O. Page de modification complète

✅ **Terminé**

Preuve	`ScenarioWizard.tsx` en mode `mode="edit"` (sections marquées « Modifié », bouton « Enregistrer » actif seulement après une modification, confirmation avant de quitter) ; route `scenarios/:id/edit` ; DTO `ScenarioFullRequest` et `ScenarioFullResponse` ; `ScenarioServiceImpl.updateFull` (transactionnel, refus 409 pendant une exécution) ; l'ancienne fenêtre de modification de `Scenarios.tsx` a été retirée.
Endpoints	`PUT /api/scenarios/{id}/full`
Tests	ScenarioFullUpdateTest (1), editPage.test (7), ExecutionControllerTest (refus 409) réussis. Vérification réelle : enregistrement en une transaction, refus d'une étape invalide sans rien modifier en base, 409 pendant une exécution.
Manque	Rien.

P. « Ajouter une étape » dans un panneau latéral ✅ Terminé

Preuve	`wiz/StepDrawer.tsx` réutilise `StepEditor` et propose « Tester cette étape » ; utilisé dans `Scenarios.tsx`.
Tests	editPage.test (7), Scenarios.test (7) réussis.
Manque	Accessibilité et thème non contrôlés visuellement dans un navigateur.

Q. Message d'authentification masqué en cas de succès ✅ Terminé

Preuve	`wiz/TestRunPanel.tsx` : le bloc d'authentification ne s'affiche plus qu'en cas d'échec.
Tests	testRun.test (6) réussi.
Manque	Rien.

R. Traçabilité VU / itération / URL finale par résultat 🟡 Partiel

Fait	Migration 020 (colonnes `execution_step_result.virtual_user`, `iteration`, `request_url`) ; `StepOutcome` et `ExecutionStepResultResponse` enrichis ; valeurs enregistrées par `ExecutionTransactionHelper` et renvoyées par l'API.
Manque	La page de détail d'une exécution (`ExecutionDetail`) n'affiche pas encore ces trois champs.

S. Options avancées d'étape dans l'éditeur 🟡 Partiel

Fait	Le backend prend en charge le timeout, le think time et le suivi des redirections au niveau de l'étape.
Manque	`StepEditor.tsx` ne contient aucun champ pour ces options (0 résultat à la recherche) : on ne peut pas les modifier depuis l'interface.

T. Annulation : conservation des résultats partiels 🟡 Partiel

Fait	L'annulation fonctionne (`POST /api/executions/{id}/cancel`, avec confirmation dans l'UI) et la fenêtre de fin affiche CANCELLED.
Manque	La finalisation ignore les exécutions qui ne sont plus RUNNING ou QUEUED : les résultats déjà mesurés sont perdus. Le texte de la fenêtre de confirmation le dit clairement.

U. Travail versionné dans Git ❌ Non réalisé

Constat	Dernier commit : `3003211` du 26/09/2026. Les 74 fichiers modifiés et les 68 entrées nouvelles ne sont pas commités.
Manque	Un ou plusieurs commits, une fois les modifications relues.

5. Endpoints REST du module

Liste extraite des annotations des contrôleurs dans le code actuel.

Méthode	Chemin	Rôle
ScenarioController — `/api/scenarios` (20)		
GET	`/api/scenarios`	Liste paginée et filtrée
POST	`/api/scenarios`	Création
GET	`/api/scenarios/{id}`	Détail
PUT	`/api/scenarios/{id}`	Mise à jour du scénario seul
PUT	`/api/scenarios/{id}/full`	Mise à jour complète, scénario et étapes
DELETE	`/api/scenarios/{id}`	Suppression
GET	`/api/scenarios/{id}/deletion-impact`	Impact d'une suppression
POST	`/api/scenarios/{id}/archive`	Archivage
POST	`/api/scenarios/{id}/restore`	Restauration
POST	`/api/scenarios/{id}/publish`	Publication d'un brouillon
DELETE	`/api/scenarios/{id}/draft`	Suppression d'un brouillon
POST	`/api/scenarios/{id}/activate`	Passage en ACTIVE
POST	`/api/scenarios/{id}/deactivate`	Passage en INACTIVE
POST	`/api/scenarios/{id}/resume-schedules`	Reprise des planifications mises en pause
POST	`/api/scenarios/bulk-deletion-impact`	Impact d'une suppression groupée
POST	`/api/scenarios/bulk-archive`	Archivage groupé
POST	`/api/scenarios/bulk-restore`	Restauration groupée
POST	`/api/scenarios/bulk-delete`	Suppression groupée
POST	`/api/scenarios/bulk-status`	Changement de statut groupé
POST	`/api/scenarios/bulk-resume-schedules`	Reprise groupée des planifications
StepController — `/api/steps` (5)		
GET	`/api/steps`	Liste (par scénario)
GET	`/api/steps/{id}`	Détail
POST	`/api/steps`	Création
PUT	`/api/steps/{id}`	Mise à jour
DELETE	`/api/steps/{id}`	Suppression
TestRunController (4)		
POST	`/api/scenarios/{id}/test-run`	Test du scénario (accepte un brouillon et `upToStep`)
POST	`/api/scenarios/{id}/test-run?stream=true`	Même test, résultat diffusé en flux NDJSON
POST	`/api/steps/{id}/test-run`	Test d'une étape
POST	`/api/steps/{id}/test-run?stream=true`	Même test, résultat diffusé en flux NDJSON
ExecutionController — `/api/executions` (9), utilisé par le module		
POST	`/api/executions`	Lancer une exécution
GET	`/api/executions`, `/history`, `/{id}`	Liste, historique, détail
GET	`/api/executions/{id}/status`	État en direct, interrogé chaque seconde
POST	`/api/executions/{id}/cancel`, `/{id}/retry`	Annulation, relance
GET	`/api/executions/{id}/report`, `/report/export`	Rapport, export

6. Base de données (migrations Liquibase)

Le projet compte 20 migrations. Hibernate tourne avec `ddl-auto: validate` : chaque colonne des entités doit exister dans le schéma créé par Liquibase. Les quatre migrations ajoutées pour ce module :

Fichier	Contenu
`017-deletion-history-and-archiving.xml`	Historique des suppressions et archivage
`018-scenario-drafts-and-csv-size.xml`	Statut DRAFT, colonnes `wizard_step` et `draft_state`, `csv_data` agrandi à VARCHAR(1000000)
`019-scenario-inactive-paused-schedules.xml`	Colonne `scheduled_execution.paused_by_scenario`
`020-variables-and-request-params.xml`	Colonnes `scenario.variables` et `csv_distribution` ; `step.path_params` et `query_params` ; `execution_step_result.virtual_user`, `iteration` et `request_url`

> Ces migrations sont appliquées et validées à chaque démarrage du contexte Spring dans les tests (base H2) : BackendApplicationTests et les tests de contrôleurs passent.

7. Résultats des tests

652
tests backend, 0 échec ni erreur

2 min 46
durée de `mvn -o test`

99
tests frontend (14 fichiers), 0 échec

11
nouvelles classes de test backend du module

7.1 Backend : classes liées au module (extrait de l'exécution du 02/10, 22:11)

Classe de test	Tests	Résultat	Origine
controller.ScenarioControllerTest	22	**22 / 22**	socle, mis à jour
controller.StepControllerTest	28	**28 / 28**	socle, mis à jour
controller.ExecutionControllerTest	23	**23 / 23**	socle, mise à jour (409)
controller.DeletionArchivingPagingTest	7	**7 / 7**	nouveau
controller.ScenarioDraftControllerTest	4	**4 / 4**	nouveau
controller.ScenarioStatusAndBulkControllerTest	5	**5 / 5**	nouveau
controller.ScenarioFullUpdateTest	1	**1 / 1**	nouveau
controller.VariablesControllerTest	1	**1 / 1**	nouveau
controller.ExecutionLiveStatusTest	1	**1 / 1**	nouveau
controller.TestRunControllerTest	4	**4 / 4**	nouveau
service.execution.StepTestRunnerTest	13	**13 / 13**	nouveau
service.execution.RequestBuilderTest	8	**8 / 8**	nouveau
service.execution.EngineVariablesTest	4	**4 / 4**	nouveau
service.execution.LiveStatsTest	2	**2 / 2**	nouveau
service.execution.CsvDataSourceTest	12	**12 / 12**	étendu
service.execution.HttpClientExecutionEngineTest	48	**48 / 48**	socle, mis à jour
service.execution.VariableResolverTest	7	**7 / 7**	socle
service.execution.JsonPathExtractorTest	11	**11 / 11**	socle
repository.ScenarioRepositoryTest / StepRepositoryTest	6 + 6	**12 / 12**	socle (Scenario… mis à jour)
Ensemble du backend (66 classes)	**652**	**652 / 652**	BUILD SUCCESS

Deux tests existants ont été adaptés à des changements de comportement voulus : le moteur échoue avant l'envoi si une variable est introuvable, au lieu d'envoyer `${token}` tel quel ; une modification pendant une exécution est refusée avec un 409.

7.2 Frontend : Vitest (02/10, 21:08)

Fichier de test	Tests	Fichier de test	Tests
Scenarios.test	7	editPage.test	7
ScenarioWizard.test	5	variables.test	8
csvModel.test	7	testRun.test	6
load.test	3	ExecutionLive.test	5
validation.test	10	statusMapping.test	10
Pagination.test	4	roleMapping.test	4
Applications.test (hors module)	13	ApplicationAuthFields.test (hors module)	10
Total			**99 / 99**

Le build de production du frontend (`tsc` puis Vite) réussit.

7.3 Vérifications réelles (backend local + banking-test-api)

- Scénario bancaire avec `docs/samples/banking-test-api-users.csv` : test avec « Tester », puis exécution à 5 VU après réinitialisation, sans erreur.
- Variables avec `docs/samples/banking-variables.csv` : 59 DELETE sur 59 `transferId` distincts, aucun 404 ni 409 inattendu. Aucun VU n'a utilisé l'identifiant d'un autre.
- Suivi en direct : statut obtenu chaque seconde pendant une exécution réelle.
- Modification complète : test d'une version non enregistrée (étapes 1 à 2) sans toucher la base ; `PUT /full` appliqué en une transaction ; étape invalide refusée sans aucune modification ; 409 pendant une exécution.

8. Ce qui manque, risques et prochaines étapes

8.1 Manques fonctionnels

Point	Statut	Ce qu'il reste à faire
Affichage VU / itération / URL finale	🟡	Ajouter les trois colonnes dans `ExecutionDetail` ; les données sont déjà dans l'API.
Timeout, think time et redirections par étape	🟡	Ajouter les champs dans `StepEditor.tsx`, avec les mêmes règles que le backend.
Résultats partiels à l'annulation	🟡	Finaliser aussi les exécutions CANCELLED pour garder les mesures déjà prises.
Commit du travail	❌	Relire puis commiter les 142 entrées de l'arbre de travail.

8.2 Limites techniques connues

- **JSONPath :** `JsonPathExtractor` ne lit que des chemins pointés (`$.a.b`). Les index de tableau (`$.items[0].id`) ne sont pas pris en charge.
- **Échappement JSON :** `RequestBuilder` injecte les valeurs telles quelles dans le corps. Une valeur CSV contenant `"` casse le JSON (constaté pendant la démonstration, contourné en changeant les données).

8.3 Anomalies et risques

- **Fichier vide en trop :** `perftest-frontend/src/ScenarioWizard.tsx` (0 octet), probablement créé par erreur par un outil. Le vrai composant est `pages/scenario-wizard/ScenarioWizard.tsx`. Ce fichier est à supprimer après confirmation.
- **Pas de contrôle visuel :** l'UI est testée par Vitest et Testing Library (jsdom), mais aucune vérification visuelle n'a été faite dans un vrai navigateur (mise en page, thème sombre, accessibilité du panneau latéral).
- **Travail non versionné :** le risque de perte est réel tant que rien n'est commité.

8.4 Prochaines étapes proposées

1. Relire et commiter le travail par lots (brouillons et CSV ; INACTIF et actions groupées ; variables ; suivi en direct ; page de modification).
2. Supprimer le fichier vide en trop.
3. Contrôler l'UI dans un navigateur, en thème clair et sombre.
4. Traiter les trois points 🟡, puis l'échappement JSON et les tableaux JSONPath.